



**COMSATS University Islamabad,
Sahiwal Campus**

GreenAgri

A Project Presented To

**COMSATS University Islamabad,
Sahiwal Campus**

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2021-2025)

By

Khurram Ali

CIIT/FA21-BCS-171/SWL

Hafiz Abdullah Munir

CIIT/FA21-BCS-166/SWL

Abdul Ahad Younas

CIIT/FA21-BCS-211/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Khurram Ali

Hafiz Abdullah Munir

Abdul Ahad Younas

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(CS) “**GreenAgri**” was developed by **Khurram Ali (CIIT/FA21-BCS-171/SWL)** , **Hafiz Abdullah Munir (CIIT/FA21-BCS-166/SWL)** and **Abdul Ahad Younas (CIIT/FA21-BCS-211/SWL)** under the supervision of “**MR. M. Arslan Sarwar**” and co supervisor “**Mr. M. Jamil**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Supervisor

Mr. M. Arslan Sarwar

Lecturer

Department of computer science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr Iqra Muneer

Assistant Professor

Department of computer science

UET Lahore

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of computer science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

The GreenAgri App is an innovative and comprehensive agricultural platform designed to facilitate seamless transactions and services for farmers, agribusinesses, and individuals in the agricultural sector. This mobile application aims to enhance agricultural trade and services by providing a user-friendly marketplace where users can effortlessly buy and sell agricultural products. The system is structured into three distinct mobile applications: the Admin App, the Seller App, and the Buyer App. The Admin App enables administrators to manage the overall system, including user accounts and platform maintenance. The Seller App is designed specifically for service providers and product sellers, allowing them to register, list agricultural products, offer rental services, and efficiently manage their offerings. Meanwhile, the Buyer App is tailored for consumers and farmers, providing them with the ability to browse and purchase listed products, access rental equipment, receive live weather updates, chat with support through an AI-powered chatbot, and stay informed with real-time agricultural news.

Key features of the Green Agri App include Product Buying & Selling, Rental Services for Agricultural Equipment, Live Weather Updates, AI-powered Chatbot Support, and Real-time Agricultural News Feeds. This project is specifically tailored for the Pakistani agricultural community, addressing the existing gaps in digital agricultural solutions by leveraging technology to optimize resource utilization, improve productivity, and promote sustainable farming practices. By combining e-commerce, AI-driven assistance, and real-time data services, Green Agri aims to empower farmers, enhance economic opportunities, and contribute significantly to the modernization of the agricultural sector.

Acknowledgment

All praise is to almighty Allah who bestowed upon us a minute portion of his boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Mr. M. Arslan Sarwar**” and our Co-Supervisor “**Mr. M. Jamil**”. Without their supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Khurram Ali

Hafiz Abdullah Munir

Abdul Ahad Younas

Abbreviations

API	Application Programming Interface
SRS	Software Require Specification
SDD	Software Design Description
GUI	Graphical User Interface

Table of Contents

1	Introduction	1
1.1	Brief overview	1
1.2	Relevance to course module.....	2
1.2.1	Mobile application development.....	2
1.2.2	Database management system	2
1.2.3	Artificial intelligence and machine learning.....	3
1.3	Project Background	3
1.4	Methodology and software Lifecycle.....	4
1.4.1	Rationale behind selected Agile Model.....	4
2	Problem Definition.....	6
2.1	Problem statement	6
2.2	Deliverable and development requirements	6
2.3	Current System.....	6
3	Requirement Analysis	8
3.1	Use case Diagram	8
3.2	Use case description.....	9
3.3	Functional Requirements.....	24
3.4	NonfunctionalRequirements	24
3.4.1	Usability	24
3.4.2	Performance	25
3.4.3	Portability	25
3.4.4	Reliability	25

4	Design and Architecture.....	27
4.1	SystemArchitecture	27
4.2	Process Flow/Representation	28
4.3	Sequence Diagram.....	29
4.4	Class Diagram.....	30
5	Data Design	32
5.1	Data Dictionary.....	32
5.2	Implementation	32
5.3	Human Interface design.....	33
6	Testing	38
6.1	Unit Testing	38
6.1.1	Buyer App unit testing.....	38
6.1.2	Seller App unit testing.....	39
6.1.3	Admin App unit testing	40
6.2	Manual Testing	41
6.2.1	Buyer App testing	41
6.2.2	Seller App testing.....	42
6.2.3	Admin App testing	43
6.3	Functional Testing	44
6.4	Integration Testing	45
7	Conclusion and Future Work.....	48
7.1	Conclusion	48
7.2	Future Work.....	48

8	References	50
---	------------------	----

Table of Tables

Table 3.1 Login	9
Table 3.2 Get Registration.....	10
Table 3.3 List Products	11
Table 3.4 List Services.....	12
Table 3.5 Set Prices	13
Table 3.6 Sell Products	14
Table 3.7 Payments.....	14
Table 3.8 Search Products.....	15
Table 3.9 Purchase Products	16
Table 3.10 Receive Products	17
Table 3.11 Payments clearance.....	18
Table 3.12 Get Services	19
Table 3.13 Manage Accounts	19
Table 3.14 Manage Products	20
Table 3.15 Manage Services	21
Table 3.16 Manage App Performance.....	22
Table 3.17 Logout	22
Table 6.1 Buyer Unit Testing	38
Table 6.2 Seller unit testing.....	39
Table 6.3 Admin unit testing.....	40
Table 6.4 Buyer app testing.....	41
Table 6.5 Seller app testing	42
Table 6.6 Admin app testing	43
Table 6.7 Functional testing	44
Table 6.8 Integration testing.....	45

Table of figures

Figure 3.1 Use Case Diagram.....	8
Figure 4.1 System Architecture	27
Figure 4.2 Process Flow.....	28
Figure 4.3 Sequence Diagram	29
Figure 4.4 Class Diagram	30
Figure 5.1 Profile.....	33
Figure 5.2 Home.....	33
Figure 5.3 Products.....	34
Figure 5.4 Services	34
Figure 5.5 Order	35
Figure 5.6 Order Status.....	35
Figure 5.7 Seller app.....	36
Figure 5.8 Admin app	36

Chapter 1

Introduction

1 Introduction

Agriculture plays a vital role in the economy of many countries, especially in rural regions where it is often the primary source of income and livelihood. Despite its importance, the agricultural sector still faces numerous challenges such as limited market access, lack of timely information, inadequate infrastructure, and poor connectivity between buyers, sellers, and service providers. In response to these issues, the Green Agri App has been developed as a smart solution to bridge the gap between agricultural stakeholders through digital technology.

The Green Agri App is a mobile application system designed to facilitate the buying and selling of agricultural products, provide rental services for farming equipment, deliver live weather updates, offer real-time news, and support users through a chatbot feature. To ensure smooth functionality and role-based access, the system is divided into three separate applications: the Admin App for managing users, services, and products; the Seller App for product and service providers to register and list their offerings; and the Buyer App for customers to browse and purchase products, access services, and stay informed.

This project aims to enhance transparency, improve service delivery, and promote digital inclusion in agriculture. By integrating various services into a single platform, the Green Agri App not only simplifies agricultural trade and communication but also empowers farmers and users with the tools and information they need to make better decisions and improve productivity. The application represents a step toward modern, tech-driven farming practices and contributes to the overall development of the agricultural sector.

1.1 Brief overview

The Green Agri App is a multi-functional mobile application system developed to support the agricultural community by offering a digital platform for buying, selling, and accessing farming-related services. Recognizing the challenges faced by farmers and agricultural stakeholders in accessing markets, services, and timely information, this project aims to bridge the gap through technology. The app ecosystem consists of three separate yet interconnected applications: the Admin App, Seller App, and Buyer App.

The Admin App is designed to provide backend control, allowing administrators to manage users, products, and service listings efficiently. The Seller App serves vendors and service providers, enabling them to register, list their agricultural products, and offer rental services such as machinery or tools. The Buyer App targets end-users who can explore the marketplace, purchase products, avail rental services, receive weather forecasts, access real-time agricultural news, and interact with a chatbot for guidance or support.

By integrating essential features into one platform, the Green Agri App fosters transparency, boosts market accessibility, and empowers farmers and users with the necessary tools and information to make informed decisions. This project contributes to the digital advancement of agriculture, aiming to create a smarter, more efficient, and connected farming ecosystem.

1.2 Relevance to course module

The Green Agri App aligns with multiple course modules from the Computer Science curriculum, integrating concepts from Android development, artificial intelligence and database management. This project serves as a practical application of theoretical knowledge gained throughout the degree program. Below are the key course modules relevant to this project.

1.2.1 Mobile application development

The Green Agri App is developed using Java/Kotlin for Android, ensuring smooth performance and user-friendly design. It follows Material Design principles and includes features like real-time updates, push notifications, and seamless navigation to enhance user experience.

1.2.2 Database management system

The app uses Firebase to manage user data, product listings, and transactions efficiently. It ensures fast data retrieval, security, and real-time synchronization, allowing seamless marketplace and service interactions.

1.2.3 Artificial intelligence and machine learning

AI is integrated into the app through an AI-powered chatbot for automated assistance and weather prediction models for real-time climate updates. These features help farmers make informed decisions for crop planning and market trends.

1.3 Project Background

Agriculture plays a crucial role in the economy of many countries, including Pakistan, where a significant percentage of the population depends on farming for their livelihood. Despite its importance, the agricultural sector faces several challenges, such as limited market accessibility, lack of technological integration, inefficient resource utilization, and unpredictable weather conditions. Farmers often struggle to find reliable marketplaces to sell their products at fair prices, access necessary equipment affordably, and obtain timely information regarding market trends and weather conditions.

With the rapid advancement of technology and the growing adoption of digital solutions in various industries, there is a pressing need to modernize agriculture through smart and accessible digital platforms. The Green Agri App is developed as a response to these challenges, aiming to provide a centralized, user-friendly platform that connects farmers, traders, and agribusinesses. By integrating e-commerce for agricultural products, rental services for equipment, real-time weather updates, AI-powered chatbot support, and agricultural news updates, Green Agri seeks to bridge the gap between traditional farming practices and modern technological advancements.

The project is inspired by the vision of smart and sustainable agriculture, where farmers have access to essential resources, market opportunities, and data-driven insights to improve productivity, reduce losses, and enhance decision-making. Green Agri is designed to empower the agricultural community, ensuring that technology serves as a tool for economic growth, efficiency, and long-term sustainability in farming.

1.4 Methodology and software Lifecycle

The development of the Green Agri App follows the Agile Software Development Methodology, which emphasizes iterative development, continuous feedback, and flexibility. Agile is a good choice for this project because it meets all of these criteria.

1.4.1 Rationale behind selected Agile Model

Agile is a software development methodology that emphasizes flexibility, collaboration, and continuous improvement. It is well suited for complex projects with changing requirements. Agile also advocates for ongoing user collaboration, which in this context means actively involving university students throughout the development process. Their feedback and evolving requirements should guide the platform's continuous improvement.

Agile is based on the following principles:

- **Iterative development:** Software is developed in cycles, with each cycle adding new functionality or improving existing functionality.
- **Incremental delivery:** Software is delivered to users in increments, with each increment adding new value.
- **Continuous feedback:** Users and stakeholders provide feedback on the software throughout the development process.
- **Adaptive planning:** The development team adapts the plan as needed to meet the needs of the users and the changing requirements.

1.5 Stakeholder Analysis

- Farmers (Buyers)
- Sellers / Service Providers
- Admins
- Development Team
- Supervisors and Evaluators
- Agricultural Institutions (optional)

Chapter 2

Problem Definition

2 Problem Definition

Identifying the challenges faced by farmers and agri-businesses in accessing digital platforms for trade, services, and real-time information.

2.1 Problem statement

Agriculture plays a crucial role in the economy, yet many farmers face challenges such as limited market access, lack of weather insights, and difficulty in renting or selling farming equipment and products. Traditional methods are inefficient, time-consuming, and lack digital connectivity. There is a need for a centralized digital platform that provides marketplace services, weather updates, rental options, and AI-driven assistance to help farmers make informed decisions and enhance productivity.

2.2 Deliverable and development requirements

The key deliverables include a fully functional Android application with modules for buying/selling products, rental services, weather updates, chatbot assistance, and real-time news. Development requirements include Java/Kotlin for Android, Firebase backend, AI integration for chatbot, and secure authentication and data storage. A user-friendly and responsive interface is also essential.

2.3 Current System

The current system in the agricultural sector is largely traditional, where farmers and service providers depend on local markets, intermediaries, and word-of-mouth to sell products or offer services. This approach is inefficient, time-consuming, and lacks transparency in pricing, availability, and quality. Buyers face difficulties in finding reliable sellers or services and often have no access to real-time weather forecasts, agricultural news, or instant assistance. Existing digital solutions are fragmented, offering only specific features like weather updates or online marketplaces without integration or role-based access. There is no unified platform that connects all stakeholders in the agriculture ecosystem, resulting in poor coordination, limited outreach, and missed opportunities for growth and development.

Chapter 3

Requirement Analysis

3 Requirement Analysis

3.1 Use case Diagram

In this Use case diagrams, we show the behavior of our system. This enable you to visualize the different types of roles in a system and how those roles interact with the system.

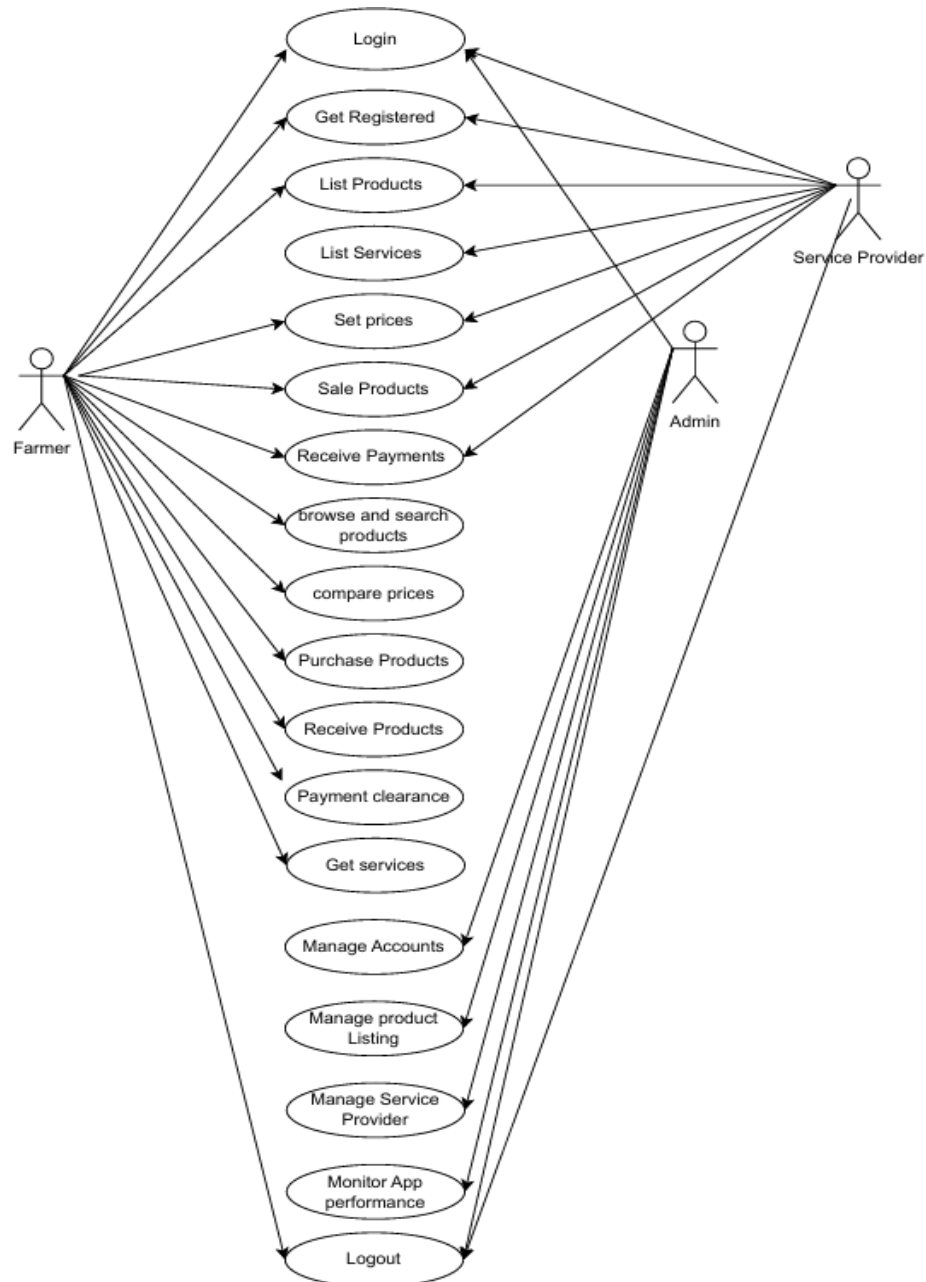


Figure 3.1 Use Case Diagram

3.2 Use case description

The tables below indicate comprehensive use case descriptions.

Table 3.1 outlines the "Login" use case for the system, involving key actors such as User, Seller, and Admin. It details the steps, conditions, and possible exceptions during the login process to ensure secure access. The use case defines both normal and alternative flows of user authentication.

Table 3.1 Login

Use case id	01
Use case name	Login
Actors	User Seller Admin
Description	A user of the System logs into the System.
Trigger	User requests to log in.
Pre-condition	The user has an account.
Postcondition	The actor is now logged into the system.
Normal flow	This use case starts when an actor wants to login into the system. The system requests that the actor enter e-mail/phone no. and password. The actor enters e-mail/phone number and password. The system validates the entered e-mail/phone no. and password and login into the system.
Alternative flow	For example, a Valid e-mail/Phone number or an Invalid Password If in the basic flow the system cannot find the valid e-mail or the password is invalid, an error message is displayed.
Exceptions	Incorrect credentials. Request to re-enter the login details.
Business rules	The system must be connected to the network.

Table 3.2 outlines the "Get Registration" use case, which involves users, sellers, or admins initiating account creation within the system. It describes the necessary steps, conditions, and flows for successful registration. The use case ensures proper validation and storage of user information for system access.

Table 3.2 Get Registration

Use case id	02
Use case name	Get Registration
Actors	Admin Seller User
Description	A user of the System creates an account.
Trigger	Select the register link
Pre-condition	When the user doesn't have an account to log in to the system.
Postcondition	The user's details are stored in the database.
Normal flow	This use case starts when the user is not logged in to the system and goes to the login page. The system prompts the user to send an e-mail and password or register a new account. User selects the registration option. The system displays the signup page that allows users to fill in their username/email address, first and last name, and password. The user enters their information. System verifies information and creates an account.

Alternative flow	Cancel Registration The users select the cancel option. The system returns the user to the home page without the user being logged in and any information entered has been erased. Invalid Information Entered Users can enter information and submit it. System displays a message to correct invalid information. The user re-enters the information.
Exceptions	If an existing user exists, then a "User already exists" message is displayed. If information is missing "Insufficient Information" message is displayed.
Business rules	Active internet access is available.

Table 3.3 describes the "List Products" use case, where users interact with the system to view available products. It details the flow of actions, conditions, and system responses, including scenarios with no available products. The use case ensures users can access product listings under valid system conditions.

Table 3.3 List Products

Use case id	03
Use case name	List products
Actors	User Seller
Description	A User lists available products in the system.
Trigger	User requests to view listed products.
Precondition	The User is logged into the system.
Postcondition	The system displays the list of products.

Normal flow	This use case starts when a farmer wants to view available products. The system retrieves and displays the list of products.
Alternative flow	If there are no products listed, a message is displayed.
Exceptions	No products available.
Business rules	The system must be connected to the network.

Table 3.4 explains the "List Services" use case, where users or sellers request to view available services in the system. It includes the necessary trigger and precondition for accessing the service list. The use case supports smooth navigation and visibility of services once the user is authenticated.

Table 3.4 List Services

Use case id	04
Use case name	List Services
Actors	User, Seller
Description	A User lists available services in the system.
Trigger	User requests to view listed services.
Precondition	The User is logged into the system.
Postcondition	The system displays the list of services.
Normal flow	The use case starts when a user (e.g., a farmer) wants to view available services. The system retrieves and displays the list of services.
Alternative flow	If there are no services listed, a message is displayed indicating this.
Exceptions	No services are available in the system.
Business rules	The system must be connected to the network.

Table 3.5 outlines the "Set Prices" use case, where sellers or buyers update product or stock prices based on their roles. It includes the normal flow of entering and validating prices, along with alternatives for invalid inputs. The use case ensures accurate price management under proper login and network conditions.

Table 3.5 Set Prices

Use case id	05
Use case name	Set Prices
Actors	Seller, Buyer
Description	A seller or buyer sets prices for listed products or stock based on their role.
Trigger	Seller or buyer requests to set or update prices.
Precondition	The user (seller or buyer) is logged into the system and has access to the relevant product or stock listing.
Postcondition	The prices of the selected items are updated in the system.
Normal flow	This use case starts when a seller or buyer wants to set or update prices. The system displays the relevant product or stock listing. The user enters or modifies the price. The system validates and saves the updated price.
Alternative flow	If the entered price is invalid (e.g., negative value, out-of-range), an error message is shown.
Exceptions	Price update failure due to server or validation errors.
Business rules	The system must be connected to the network.

Table 3.6 explains the "Sell Products" use case, where farmers or shopkeepers list their products for sale in the system. It details the process of entering product information, validation, and handling invalid or incomplete entries. The use case ensures smooth product listing under valid login and network conditions.

Table 3.6 Sell Products

Use case id	06
Use case name	Sell Products
Actors	user Seller
Description	A farmer lists their products for sale in the system. A Shopkeeper lists their products for sale in the system.
Trigger	User requests to sell a product.
Pre-condition	The User is logged into the system.
Postcondition	The product is listed for sale in the system.
Normal flow	This use case starts when a farmer wants to sell a product. The system requests details about the product (type, quantity, price). The User enters the required information. The system validates the information and lists the product for sale.
Alternative flow	If the information is incomplete or invalid, an error message is displayed.
Exceptions	Invalid product details. Request to re-enter product details.
Business rules	The system must be connected to the network.

Table 3.7 outlines the "Payments" use case, where sellers (farmers or shopkeepers) receive payments after a successful purchase. It covers the standard payment process, system notifications, and handling of failed transactions. The use case ensures secure and recorded payment processing under proper conditions.

Table 3.7 Payments

Use case id	07
Use case name	Payments

Actors	Seller (Farmer/shopkeeper)
Description	A seller receives payments for sold products.
Trigger	Buyer completes the purchase.
Precondition	The seller has sold products.
Postcondition	The payment is received and recorded in the system.
Normal flow	This use case starts when a buyer completes the purchase. The system processes the payment. The seller receives a notification of the payment. The payment is recorded in the system.
Alternative flow	If the payment fails, an error message is displayed.
Exceptions	Payment processing error. Request to retry the payment.
Business rules	The system must be connected to the network.

Table 3.8 presents the "Search Products" use case, where users (farmers or buyers) browse available products in the system. It includes the standard flow of displaying products and an alternative for when none are available. The use case ensures product search functionality under valid login and network conditions.

Table 3.8 Search Products

Use case id	08
Use case name	Search Products
Actors	User (Farmer/Buyer)
Description	A user Search the products available in the system.
Trigger	User requests to browse products.
Precondition	The user is logged into the system.
Postcondition	The user views the available products.

Normal flow	This use case starts when a user wants to browse products. The system displays the list of available products.
Alternative flow	If no products are available, a message is displayed.
Exceptions	No products available.
Business rules	The system must be connected to the network.

Table 3.9 explains the "Purchase Products" use case, where a buyer selects items and confirms their purchase from the system. It outlines the standard purchase process, handling of out-of-stock items, and exceptions like network errors. The use case ensures smooth and validated purchasing under proper conditions.

Table 3.9 Purchase Products

Use case id	09
Use case name	Purchase Products
Actors	Buyer
Description	A buyer purchases products listed by sellers in the system.
Trigger	Buyer selects products and confirms the purchase
Precondition	Buyer is logged in and has selected products in the cart.
Postcondition	The purchase is completed, and the system initiates the payment process.
Normal flow	This use case starts when a buyer selects products and proceeds to checkout. The system shows a summary of the selected products. The buyer confirms the purchase. The system triggers the payment process

Alternative flow	If the product is out of stock or unavailable, the system informs the buyer and removes it from the list.
Exceptions	Purchase cannot proceed due to network errors, product unavailability, or validation failures.
Business rules	The system must be connected to the network. Only available products can be purchased.

Table 3.10 outlines the "Receive Products" use case, where the buyer receives products after a successful purchase and shipment. It details the standard delivery confirmation process and alternatives like delays or complaints. The use case ensures accurate tracking and completion of orders under valid network conditions.

Table 3.10 Receive Products

Use case id	10
Use case name	Receive Products
Actors	Buyer
Description	A buyer receives the products after a successful purchase
Trigger	The seller ships the product and marks it as delivered.
Precondition	The buyer has completed the purchase and the product is shipped.
Postcondition	The buyer receives the product and the system updates delivery status.
Normal flow	This use case starts when a seller ships the purchased product. The system tracks the delivery status. Once the buyer receives the product, the delivery is confirmed. The system marks the order as completed.
Alternative flow	If the product is delayed or not received, the buyer can raise a complaint or request support.
Exceptions	Lost shipment, incorrect delivery, or delivery system failure.
Business rules	The system must be connected to the network.

Table 3.11 describes the "Payment Clearance" use case, where buyers check the status of their payments after a transaction. It includes the flow of accessing payment history, viewing statuses, and handling pending or failed payments. The use case ensures transparent tracking of payment outcomes under proper login and system connectivity.

Table 3.11 Payments clearance

Use case id	11
Use case name	Payment clearance
Actors	Buyer
Description	A buyer checks and verifies the status of their payment after purchasing a product or service.
Trigger	Buyer navigates to order history or payment section to check payment status.
Precondition	Buyer is logged into the system and has made a purchase.
Postcondition	The payment status (cleared, or failed) is displayed to the buyer.
Normal flow	This use case starts when the buyer accesses their order or payment history. The system fetches payment records for recent transactions. The buyer selects a specific transaction. The system displays the payment status.
Alternative flow	If the payment is still pending, the system offers an option to retry or contact support.
Exceptions	Network or server error Invalid transaction ID
Business rules	The system must be connected to the network.

Table 3.12 outlines the "Get Services" use case, where a user (farmer) requests agricultural services through the system. It details the standard flow of selecting and scheduling a service, along with handling unavailability or scheduling issues. The use case ensures timely processing

of service requests under valid login and network conditions.

Table 3.12 Get Services

Use case id	12
Use case name	Get Services
Actors	User (Farmer)
Description	A user requests agricultural service.
Trigger	User requests a service.
Precondition	The user is logged into the system.
Postcondition	The service request is recorded and processed.
Normal flow	This use case starts when a user requests a service. The system displays available services. The user selects a service. The system processes the request and schedules the service.
Alternative flow	If the service is unavailable, a message is displayed.
Exceptions	Service scheduling issues. Request to reschedule the service.
Business rules	The system must be connected to the network.

Table 3.13 explains the "Manage Accounts" use case, where the admin performs actions such as creating, updating, or deleting user accounts. It outlines the standard account management flow and addresses invalid data or system errors. The use case ensures efficient user administration under valid login and network conditions.

Table 3.13 Manage Accounts

Use case id	13
Use case name	Manage Accounts
Actors	Admin
Description	Admin manages user accounts in the system.
Trigger	Admin requests to manage accounts.
Precondition	The admin is logged into the system.
Postcondition	The user accounts are managed (created, updated, deleted).

Normal flow	This use case starts when an admin wants to manage user accounts. The system displays the list of user accounts. The admin selects an account to create, update, or delete. The system processes the request and updates the accounts.
Alternative flow	If the account details are invalid, an error message is displayed.
Exceptions	Account management issues.
Business rules	The system must be connected to the network.

Table 3.14 presents the "Manage Products" use case, where the admin oversees the creation, modification, or removal of product listings. It includes the standard flow of managing listings and handling errors due to invalid inputs or system issues. The use case ensures accurate product management under valid login and network conditions.

Table 3.14 Manage Products

Use case id	14
Use case name	Manage Products
Actors	Admin
Description	Admin manages the product listings in the system.
Trigger	Admin requests to manage product listings.
Precondition	The admin is logged into the system.
Postcondition	The product listings are managed (created, updated, deleted).
Normal flow	This use case starts when an admin wants to manage product listings. The system displays the list of product listings. The admin selects a listing to create, update, or delete. The system processes the request and updates the listings.
Alternative flow	If the listing details are invalid, an error message is displayed.
Exceptions	Listing management issues.

Business rules	The system must be connected to the network.
----------------	--

Table 3.15 outlines the "Manage Services" use case, where the admin handles the creation, update, or deletion of service listings in the system. It covers the normal management flow along with alternatives for invalid input and system exceptions. The use case ensures proper service listing management under secure login and active network conditions.

Table 3.15 Manage Services

Use case id	15
Use case name	Manage Services
Actors	Admin
Description	Admin manages the Services listings in the system.
Trigger	Admin requests to manage Services listings.
Precondition	The admin is logged into the system.
Postcondition	The Services listings are managed (created, updated, deleted).
Normal flow	This use case starts when an admin wants to manage services listings. The system displays the list of services listings. The admin selects a listing to create, update, or delete. The system processes the request and updates the listings.
Alternative flow	If the listing details are invalid, an error message is displayed.
Exceptions	Listing management issues.
Business rules	The system must be connected to the network.

Table 3.16 describes the "Manage App Performance" use case, where the admin monitors system metrics and takes actions to maintain or enhance app performance. It includes the flow of viewing metrics, identifying issues, and resolving them. The use case ensures efficient performance management under valid login and network conditions.

Table 3.16 Manage App Performance

Use case id	16
Use case name	Manage App Performance
Actors	Admin
Description	Admin monitors and manages the app's performance.
Trigger	Admin requests to manage app performance.
Precondition	The admin is logged into the system.
Postcondition	The app's performance is managed and optimized.
Normal flow	This use case starts when an admin wants to manage app performance. The system displays performance metrics. The admin analyzes the metrics and takes necessary actions.
Alternative flow	If performance metrics are unavailable, a message is displayed.
Exceptions	Performance management issues.
Business rules	The system must be connected to the network.

Table 3.17 explains the "Log Out" use case, where users (farmers, sellers, or admins) securely exit the system. It covers the logout confirmation, redirection to the login screen, and handling of interruptions or session clearance failures. The use case ensures secure session termination and proper navigation under active login conditions.

Table 3.17 Logout

Use case id	17
Use case name	Log Out
Actors	User (Farmer)/seller/Admin
Description	Users logs out of the system.
Trigger	Users selects the logout option.
Precondition	The users is logged into the system.
Postcondition	The users is logged out and redirected to the login screen.

Normal flow	The use case starts when users selects the logout option. The system prompts the users to confirm the action. Upon confirmation, the system logs the users out. The user is redirected to the login screen.
Alternative flow	If the logout process is interrupted, the system displays an error message.
Exceptions	System fails to clear the session.
Business rules	User session data must be cleared upon logout.

3.3 Functional Requirements

The agriculture app comprises three key modules: the Farmer Module, Admin Module, and Seller Module. The Farmer Module allows farmers to register, log in, list and sell products, negotiate prices, receive payments, and make purchases. It also enables them to compare product prices, receive weather and market updates, and access agricultural services such as tractor rentals, labor, equipment, and chatbot support. The Admin Module facilitates the management of user accounts, product listings, and service providers, while also enabling the updating of market information and monitoring of app performance. The Seller Module allows service providers to register, log in, list products and services such as repairs, labor, and rental equipment, manage their offerings, and handle bookings, ensuring seamless connectivity with farmers in need of their services.

3.4 Nonfunctional Requirements

- Non-Functional Requirements are the quality or abstract qualities that should be in your system.
- Non-functional requirements explain the constraints of the system. It identifies the attributes under which the system is operating.
- Some of the non-functional requirements for Agriculture App are given below.

3.4.1 Usability

- The agriculture app is designed for high usability, featuring an intuitive interface for easy navigation across its modules.
- It provides tailored functionalities for farmers, admins, and service providers, facilitating seamless registration, login, product management, and service bookings. Farmers can quickly list, sell, buy, and negotiate products, and access real-time

weather and market updates to make informed decisions. Service providers can efficiently offer and manage their agricultural services. Admins benefit from streamlined account management, inquiry responses, and app performance monitoring.

3.4.2 Performance

The agriculture app is engineered for robust performance, ensuring quick and reliable operations across all modules. It leverages efficient backend processes to handle high volumes of transactions and user interactions smoothly. Real-time data updates and low latency responses provide users with up-to-date information and seamless experiences. Continuous monitoring and optimization by admins ensure that the app remains responsive and efficient, providing a dependable platform for all users.

3.4.3 Portability

The Green Agri app is designed specifically for Android smartphones, ensuring seamless performance and user experience on a wide range of Android devices. Its responsive design adapts effectively to different screen sizes and resolutions, providing a consistent and intuitive interface across various Android models. Although the app does not support other operating systems or web browsers, its optimized mobile-first approach ensures that farmers and users can access all features and functionalities directly from their Android phones, enhancing convenience and accessibility in rural and urban settings alike.

3.4.4 Reliability

The agriculture app is designed for high reliability, ensuring consistent and dependable performance. Robust backend infrastructure supports stable operation even under heavy user loads and high transaction volumes. Regular maintenance and updates help prevent downtime and address potential issues promptly. The app incorporates failover mechanisms and data redundancy to minimize the risk of data loss and ensure continuous availability. Comprehensive testing is conducted to identify and resolve bugs, ensuring that the app functions correctly under various conditions. Users can rely on the app for accurate, up-to-date information and seamless interactions, making it a trustworthy tool for their agricultural needs.

Chapter 4

Design and Architecture

4 Design and Architecture

4.1 System Architecture

The app follows a client-server architecture. The mobile application interacts with a backend server to manage product listings, transactions, user data, and real-time market/weather information. The backend is built using a microservice architecture to handle different tasks such as transaction processing, weather data, and user management.

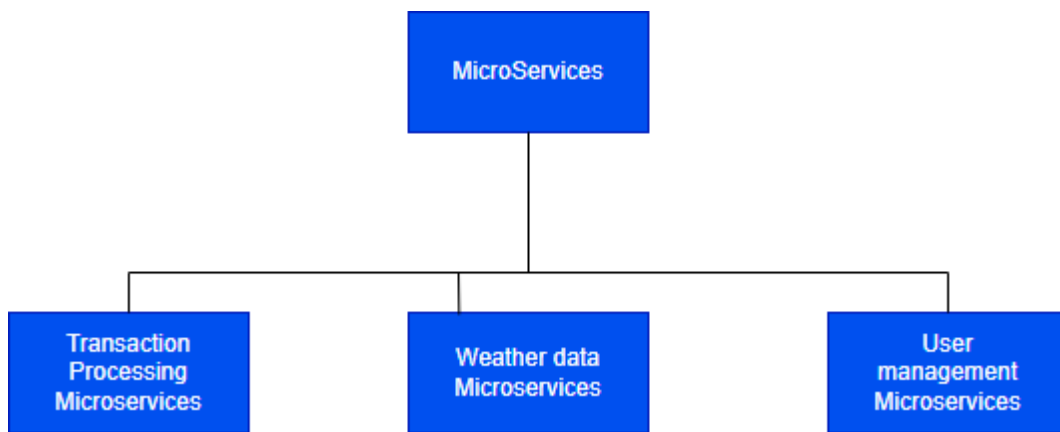


Figure 4.1 System Architecture

4.2 Process Flow/Representation

The Process Flow image visually represents the interaction processes for three main user roles in the GreenAgri system: User (Farmer), Seller, and Admin. Each role follows a distinct sequence of steps starting from registration and login to accessing specific functionalities like viewing products, booking services, listing products, and managing the platform. The User section allows service booking and product purchases; the Seller section handles service listing and product selling; and the Admin section focuses on system management tasks. All modules are interconnected via a common payment and booking confirmation flow.

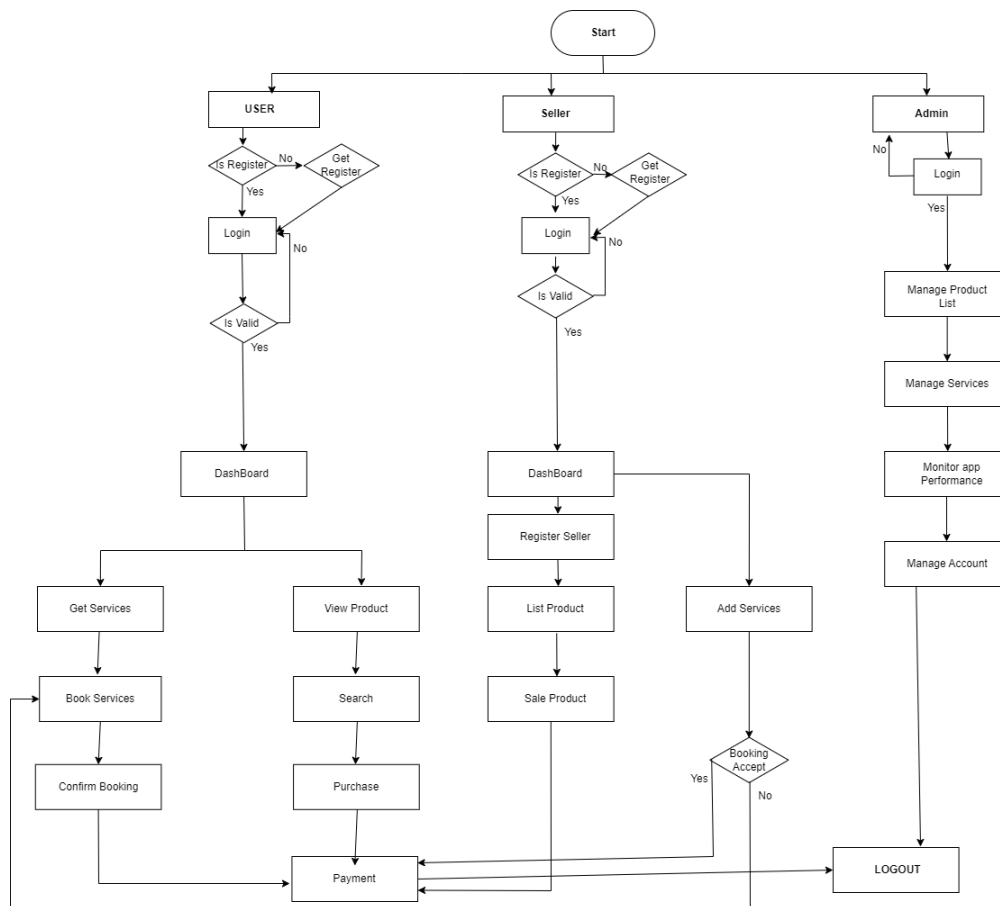


Figure 4.2 Process Flow

4.3 Sequence Diagram

These figure shows how the system will work through sequence diagrams, task after task sequence wise. It gives a complete description of the working of GreenAgri App.

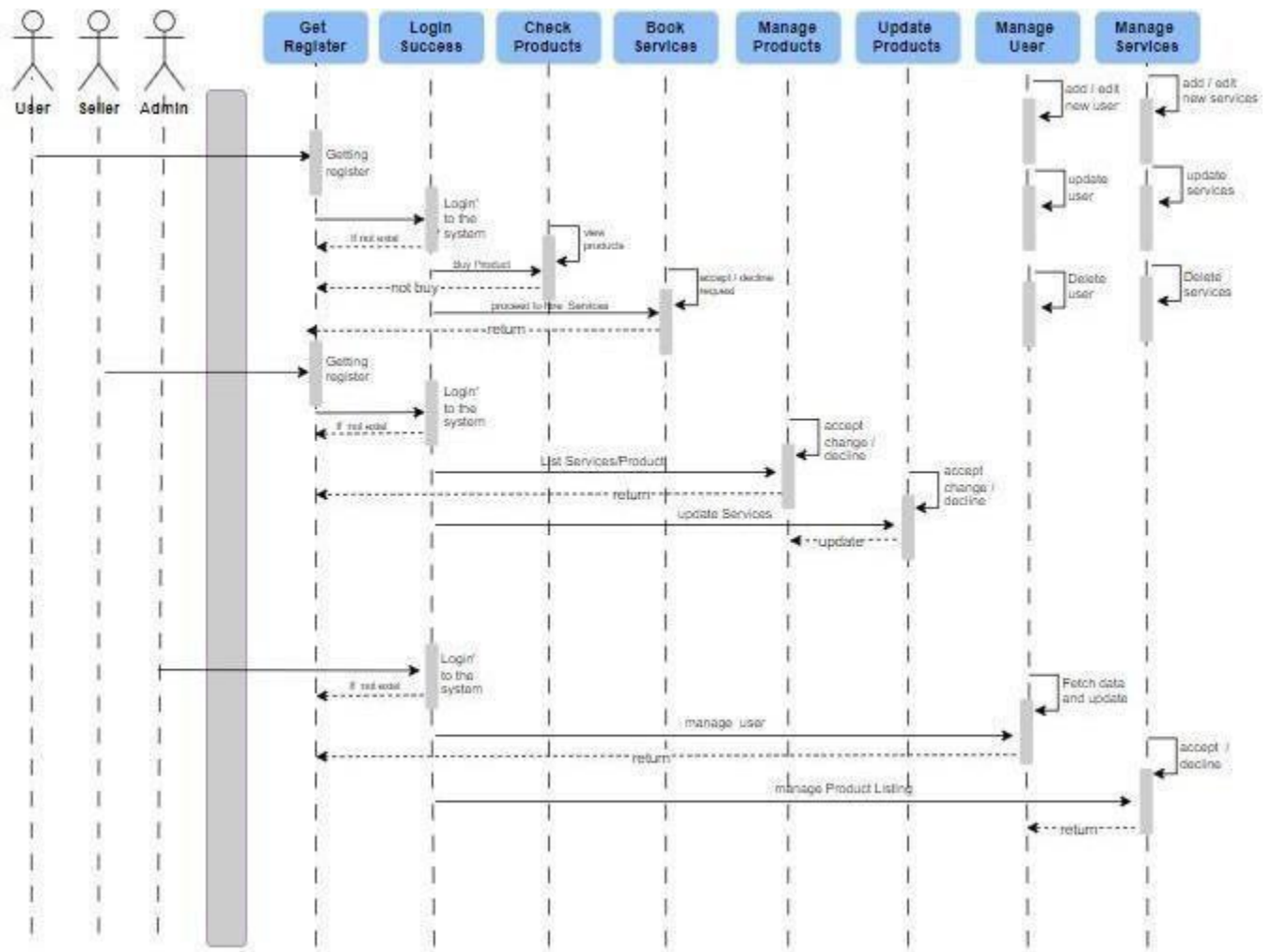


Figure 4.3 Sequence Diagram

4.4 Class Diagram

This diagram is a Class Diagram representing the structural design of the GreenAgri system. It shows the main classes involved, such as User, Seller, Admin, Product, Booking, Payment, and Announcement, along with their attributes and methods. The relationships (1-to-1, 1-to-many, many-to-many) between these entities highlight how users interact with products and services, how sellers manage offerings, how admins control system content, and how bookings and payments are processed. Each class encapsulates specific functionalities relevant to its role in the system.

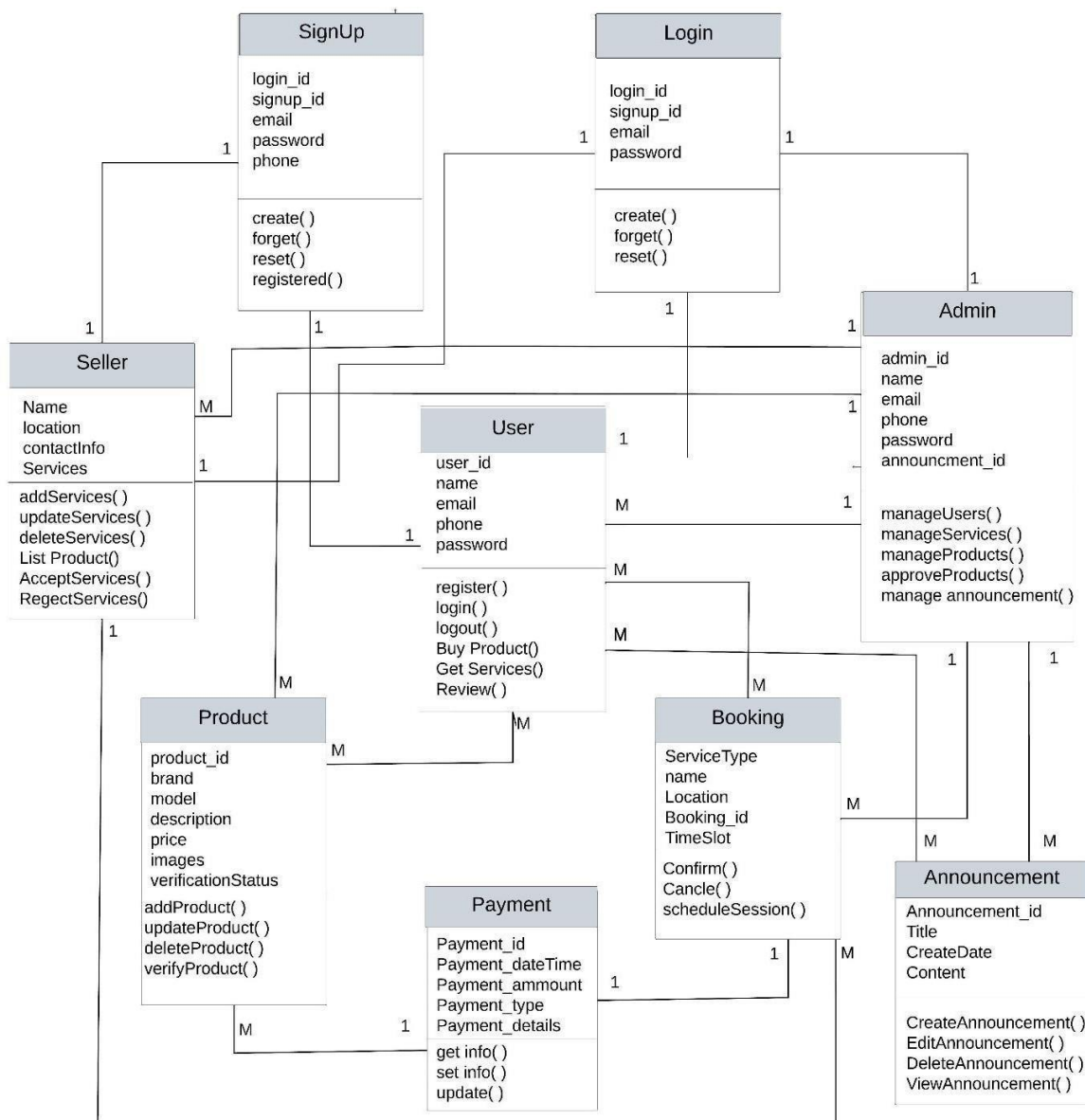


Figure 4.4 Class Diagram

Chapter 5

Data Design and Implementation

5 Data Design

The app will handle a variety of data, including user information, product listings, transaction records, and weather updates. Below is a breakdown of key data entities:

- **Farmer Information:** Name, contact details, location, product listings.
- **Product Information:** Product type, price, quantity, and location.
- **Transaction Information:** Payment details, product status.
- **Service Provider Information:** Service type, pricing, availability.
- **Weather/Market Data:** Real-time updates provided via APIs.

5.1 Data Dictionary

Data Dictionary of GreenAgri is the list of the names and attributes that are being used in the project. These are following.

- **Farmer:** User who registers and participates in buying/selling.
- **Product:** Agricultural product listed by farmers for sale.
- **Transaction:** A record of a sale or purchase.
- **Service Provider:** Providers offering agricultural services (labor, machinery).
- **Weather/Market Update:** Real-time weather and price information.

5.2 Implementation

- **Product Listing Algorithm:** Handles product creation, categorization, and visibility.
- **Transaction Algorithm:** Manages payment flow, confirmation, and receipt generation.
- **Weather API Integration:** Fetches real-time weather data based on the user's location.
- **Market Price Algorithm:** Retrieves and updates market price information for different products.

The app will be developed using Kotlin (for Android development) and will integrate API for weather Update. The backend will be built with Kotlin and Firebase for real-time data management and user authentication.

5.3 Human Interface design

Human Interface Design is a design that is visible to user. It should be user friendly and intuitive to ensure the seamless experience for our customers. Here are following some design of Green Agri System.

1. Profile Screen:

The profile screen allows users to view and manage their personal details and account settings. It provides options related to user activity, preferences, and logout.

2. Home Screen (Discounted & Trending Products):

This screen displays discounted and trending agricultural products for easy browsing. Users can explore items and navigate through the app using the bottom menu.

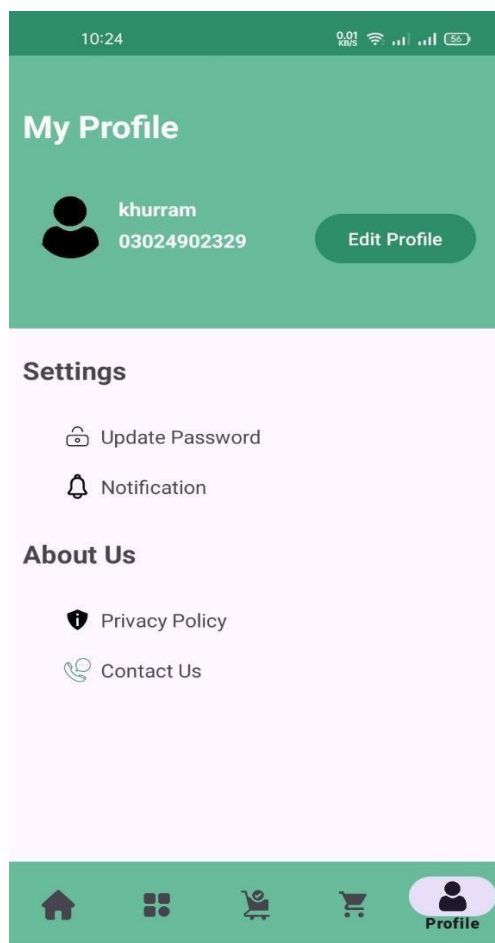


Figure 5.2 Profile

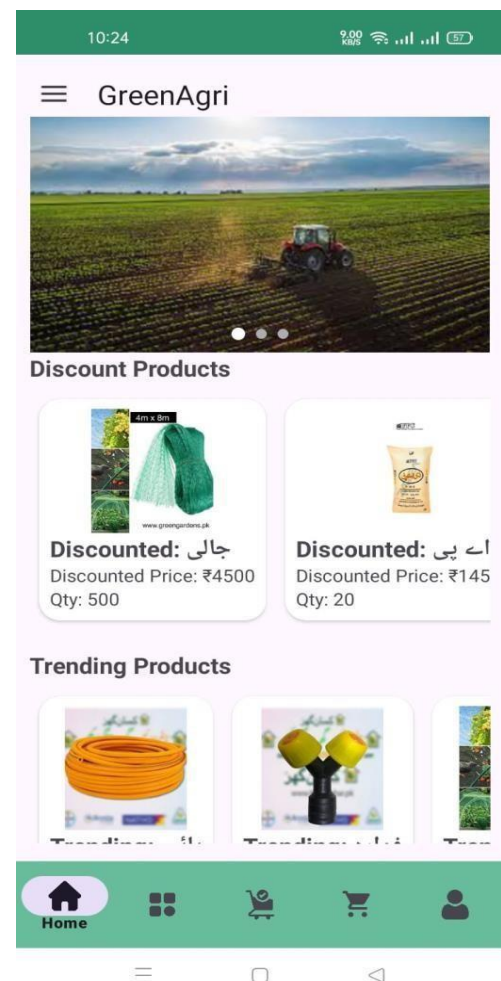


Figure 5.1 Home

Categories Screen:

This screen displays agricultural items under Products (e.g., Seeds, Sprays, Fertilizers, Machinery) and Services (e.g., rental equipment, labor service, chatbot support, agriculture news, weather updates, and repair services for farmers).

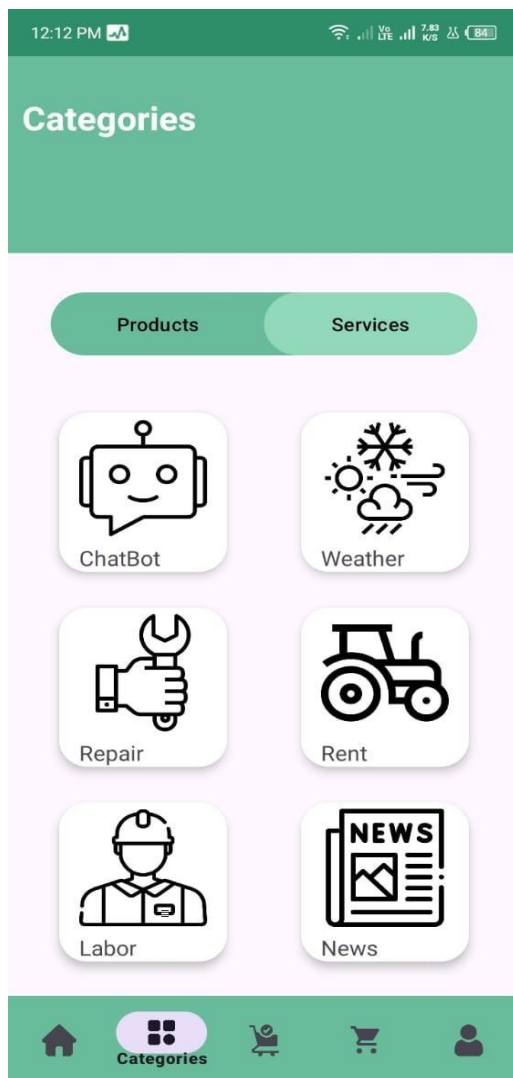


Figure 5.4 Services

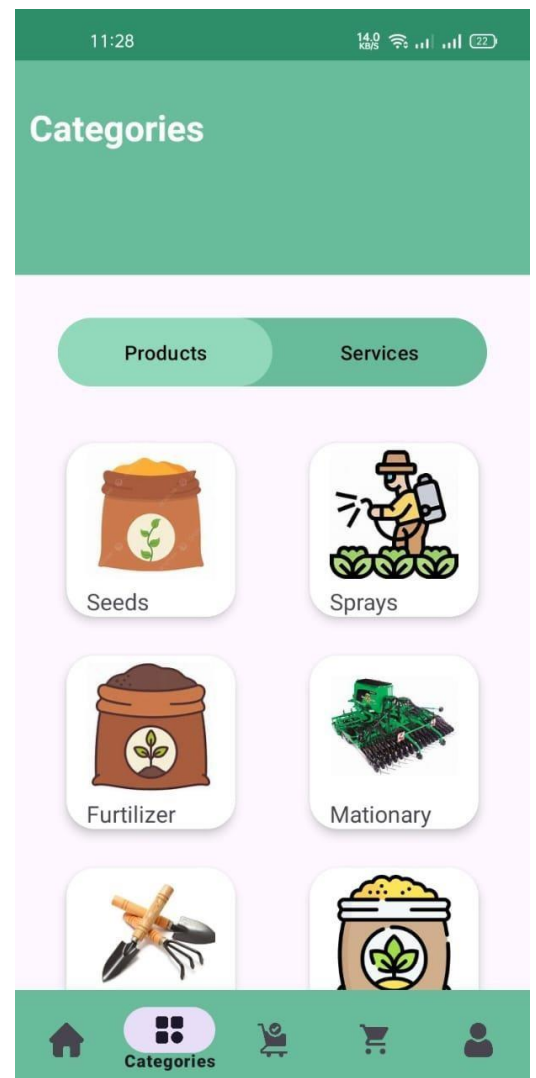


Figure 5.3 Products

1. Order View Screen:

This screen displays a list of all orders placed by the user, showing basic details like product name, quantity, and price. It allows users to review their past and current orders in one place.

2. Order Status Screen:

This screen shows the **status** of each order such as **Pending** or **Delivered**—along with relevant order details. It helps users track the progress and delivery of their purchases.



Figure 5.6 Order

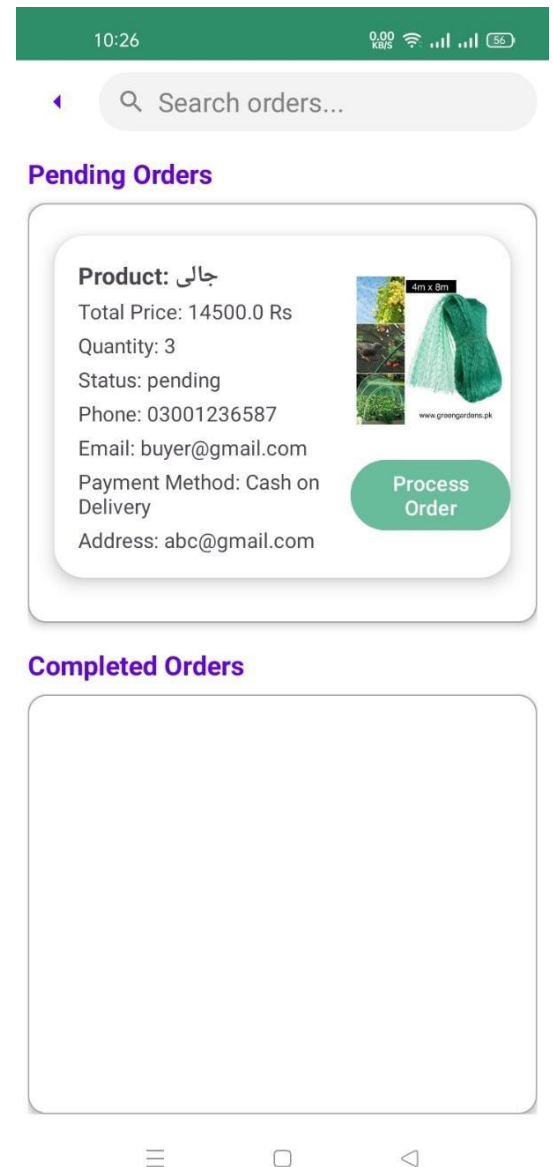


Figure 5.5 Order Status

1. Seller App Home Screen:

This is the Seller Dashboard, where sellers can add products, list services, update stock, and **view orders**. It provides a central hub for managing all seller-related tasks in a user-friendly layout.

2. GreenAgri Admin App Screen:

This is the Admin Dashboard of the GreenAgri app, offering quick access to manage buyers, sellers, deliveries, feedback, performance, and social media. The admin can monitor activities and ensure smooth system operation.

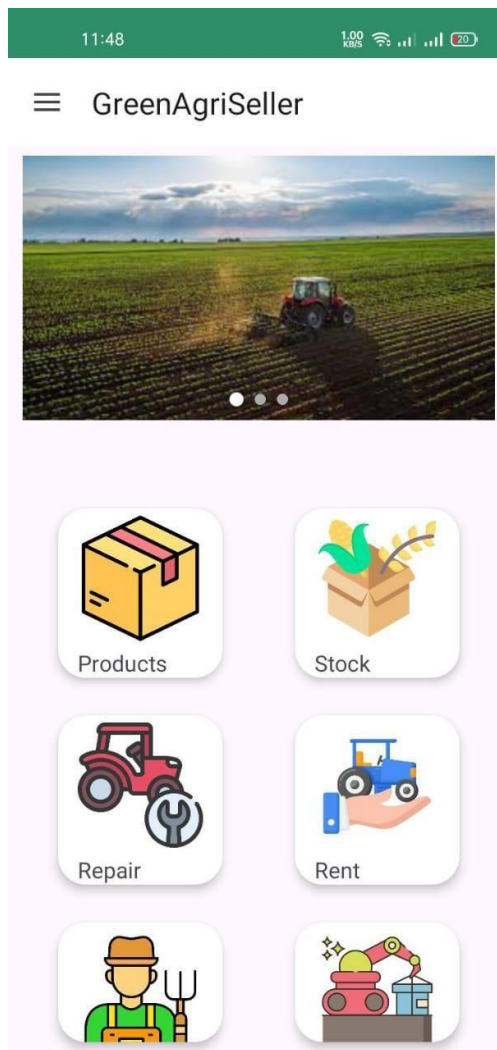


Figure 5.8 Seller app

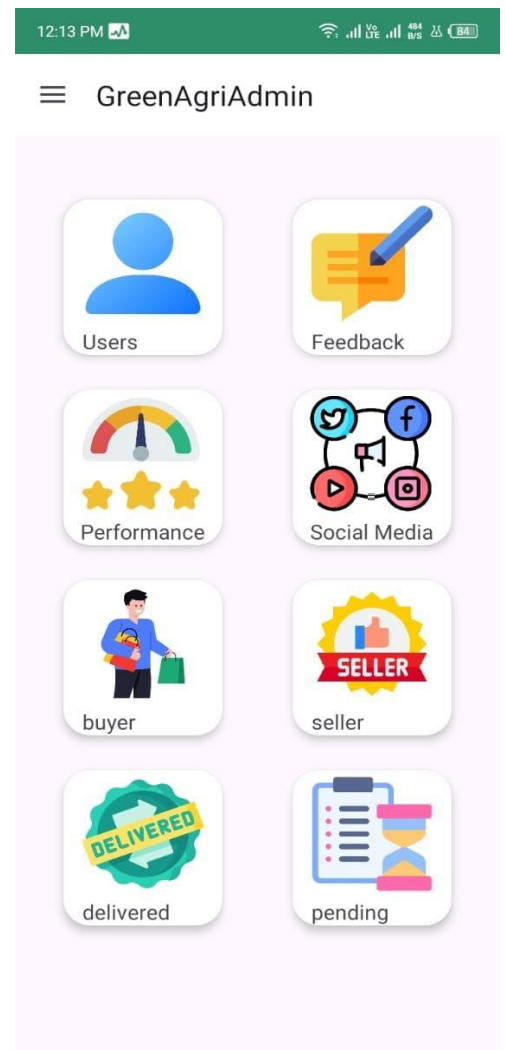


Figure 5.7 Admin app

Chapter 6

Testing and Evaluation

6 Testing

6.1 Unit Testing

Unit tests is conducted to assess the individual components of the GreenAgri App.

6.1.1 Buyer App unit testing

Table 6.1 Buyer Unit Testing

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	User Registration	Valid user information	User account is created successfully	Pass
UT-002	User Authentication	Valid user credentials	Successful authentication and user login	Pass
UT-003	View Product Listings	Fetch product list	All products are displayed	Pass
UT-004	View Services	Fetch service list	All services are displayed	Pass
UT-005	Search Functionality	Search keyword	Matching products/services displayed	Pass
UT-006	Chatbot Query	Ask a question	Relevant farming suggestion displayed	Pass
UT-007	Logout	Logout button	User is logged out	Pass

6.1.2 Seller App unit testing

Table 6.2 Seller unit testing

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-S001	Seller Login	Valid credentials	Seller logs in successfully	Pass
UT-S002	Add Product	Product name, price, category	Product is listed	Pass
UT-S003	Add Service	Service name, details	Service is listed	Pass
UT-S004	Edit Listing	Modify existing product/service	Listing is updated	Pass
UT-S005	Delete Listing	Delete request	Listing is removed	Pass
UT-S006	View My Listings	Fetch seller's items	Listings displayed correctly	Pass
UT-S007	Profile Management	Update name, image, info	Profile updated	Pass
UT-S008	Logout	Logout button	Seller is logged out	Pass

6.1.3 Admin App unit testing

Table 6.3 Admin unit testing

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-A001	Admin Login	Valid credentials	Admin logs in successfully	Pass
UT-A002	View User Data	Fetch all user info	All user records displayed	Pass
UT-A003	View Listings	All product/service data	Listings displayed	Pass
UT-A004	View Sold Products	Fetch sales data	Sold items shown correctly	Pass
UT-A005	Feedback Management	Fetch feedback	Feedback displayed clearly	Pass
UT-A006	Shopkeeper Info	Fetch shopkeeper records	Shopkeeper data displayed	Pass
UT-A007	Logout	Logout button	Admin is logged out	Pass

6.2 Manual Testing

During the manual testing phase, we rigorously assessed "GreenAgri App" for various functionalities. Any issues identified during testing were promptly addressed to guarantee a seamless user experience within the application.

6.2.1 Buyer App testing

Table 6.4 Buyer app testing

Test Cases	Expected Result	Actual Result	Pass/Fail
App Launch	Successful	Successful	Pass
User Registration	Successful	Successful	Pass
User Login	Successful	Successful	Pass
View Product Listings	Accurate	Accurate	Pass
View Service Listings	Accurate	Accurate	Pass
Search Products/Services	Relevant Results Displayed	Relevant Results Displayed	Pass
View Weather Forecast	Accurate Forecast Info	Accurate Forecast Info	Pass
Ask Farming Query (Chatbot)	Relevant Response	Relevant Response	Pass
Manage Profile	Successful	Successful	Pass
Logout	Successful	Successful	Pass

6.2.2 Seller App testing

Table 6.5 Seller app testing

Test Cases	Expected Result	Actual Result	Pass/Fail
App Launch	Successful	Successful	Pass
User Registration	Successful	Successful	Pass
User Login	Successful	Successful	Pass
Add Product Listing	Product Saved Successfully	Product Saved Successfully	Pass
Add Service Listing	Service Saved Successfully	Service Saved Successfully	Pass
Edit/Delete Listings	Changes Reflected	Changes Reflected	Pass
View My Listings	Accurate Display	Accurate Display	
Manage Profile	Successful	Successful	Pass
Logout	Successful	Successful	Pass

6.2.3 Admin App testing

Table 6.6 Admin app testing

Test Cases	Expected Result	Actual Result	Pass/Fail
App Launch	Successful	Successful	Pass
User Login	Successful	Successful	Pass
View Registered Users	Accurate User Data	Accurate User Data	Pass
View Product/Service Listings	All Listings Displayed	All Listings Displayed	Pass
View Shopkeeper Data	Accurate Records	Accurate Records	Pass
View User Feedback	Feedback Displayed Clearly	Feedback Displayed Clearly	Pass
Logout	Successful	Successful	Pass

6.3 Functional Testing

Functional testing verifies End to end functionalities of a GreenAgri App against its functional requirements and specifications, to ensure that the software behaves as expected.

Table 6.7 Functional testing

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
FT-001	Buyer Registration/Logi n	Registering or logging in with valid data	Buyer account is created or accessed successfully	Pass
Ft-002	Seller Login	Logging in as a seller with valid credentials	Seller is authenticated and navigated to dashboard	Pass
FT-003	Product Listing (Seller)	Adding a product with valid details	Product is listed and visible to buyers	Pass
FT-004	Service Listing (Seller)	Adding a service entry	Service appears in buyer app	Pass
FT-005	View Listings (Buyer)	Buyer views available products/services	Accurate listings are displayed	Pass
FT-006	Product/Service Search (Buyer)	Using keywords in search bar	Relevant results appear based on search input	Pass
FT-007	Chatbot Functionality (Buyer)	Asking farming-related questions	Appropriate responses are returned	Pass
FT-008	Profile Management (Seller)	Editing name, contact info, or image	Profile updates are saved and reflected	Pass
FT-009	View Sold Products (Admin)	Accessing list of sold items	Sold items data is correctly displayed	Pass

FT-010	Feedback Review (Admin)	Viewing feedback submitted by users	Feedback is clearly displayed and categorized	Pass
FT-011	Shopkeeper Data (Admin)	Accessing all shopkeeper profiles and activities	Complete and accurate information is shown	Pass

6.4 Integration Testing

Integration testing is the process of verifying that different modules or components of your application work together as expected

Table 6.8 Integration testing

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
IT-01	Firebase Authentication Integration	User login and registration across all apps (Buyer, Seller)	Users authenticated successfully	Pass
IT-02	Product Listing Sync	Product/service data added by Seller displayed to Buyer	Listings synced and shown in Buyer app	Pass
IT-03	Chatbot API Integration	Query sent to chatbot and receive farming advice	Response received and displayed to user	Pass
IT-04	Speech-to-Text Integration	Voice input processed and used as chatbot input	Voice converted and processed correctly	Pass
IT-05	Text-to-Speech Integration	Chatbot response converted to audio	Audio output played correctly	Pass

IT-06	Admin Dashboard Integration	Admin views sold products, users, feedback, and listings	Data from all apps fetched and displayed	Pass
IT-07	Weather API Integration	Fetch weather data and display on weather screen	Real-time weather displayed correctly	Pass
IT-08	Feedback Flow	Feedback submitted by Buyer visible to Admin	Feedback received and shown in Admin panel	Pass
IT-09	Product Status Update	Product sold status updated from Seller to Admin	Status reflected in Admin's view	Pass
IT-010	Database Connectivity	Cloud Firestore/Realtime DB operations across apps	Data saved, retrieved, and synced correctly	Pass

Chapter 7

Conclusion and Future Work

7 Conclusion and Future Work

7.1 Conclusion

The **Green Agri** project represents a significant advancement in supporting and empowering the agricultural community through technology. Its development and implementation have addressed several key challenges faced by farmers, including access to weather updates, market information, product rentals, and expert farming advice. The app provides a specialized, user-centric, and mobile-friendly platform tailored to the real-world needs of Pakistani farmers.

With its intuitive interface, the Green Agri app ensures that users can effortlessly navigate through various features such as weather forecasts, chatbot-based farming support, product listings, and equipment rentals. These functionalities have contributed to better decision-making, increased productivity, and a more connected agricultural ecosystem.

As we conclude this project, it is evident that Green Agri has successfully met its objectives by offering a practical, accessible, and farmer-focused digital solution. The app's functionality, relevance, and mobile compatibility make it a valuable tool for enhancing the livelihoods of farmers and promoting smarter agriculture in Pakistan.

7.2 Future Work

While the initial development of **Green Agri** has been successful in delivering core features to support farmers and agricultural stakeholders, there are several areas identified for future enhancement and growth:

- **Plant Disease Detection via Camera:**

Integrating AI-powered image recognition to allow users to detect plant diseases by simply capturing images of affected crops.

- **Offline Functionality:**

Implementing offline access for essential features like crop tips, weather updates, and saved chatbot queries to assist users in low-connectivity areas.

- **Expanded Language Support:**
Adding support for local languages such as Urdu, Punjabi, and Sindhi to make the app more accessible to diverse user groups.
- **In-App Notifications:**
Introducing push notifications to alert users about weather warnings, market trends, or new product listings.
- **Payment Gateway Integration:**
Enabling secure and seamless in-app transactions for buying/selling agricultural products or renting equipment.
- **Admin Panel Enhancements:**
Expanding the admin interface to allow better control over user management, content moderation, and analytics.
- **User Training and Awareness:**
Providing onboarding tutorials, help sections, and video guides to ensure users understand and utilize all available features effectively.

8 References

- [1] <https://www.farmbrite.com/>
- [2] <https://www.asiafarming.com/22-agricultural-apps-for-smart-farming-solutions- help- with-farm-management-soil-and->
- [3] <https://indidime.com/blog/best-agricultural-mobile-apps-for-farmers>
- [4] https://lucid.app/lucidspark/407876b9-40c7-48c8-bc0b-1fbca48214c7/edit?viewport_loc=-4060%2C-24562%2C22625%2C10832%2C0_0&invitationId=inv_82370c05-f5cd-46a1-8d7f-a6e80739d4c4
- [5] https://lucid.app/lucidspark/407876b9-40c7-48c8-bc0b-1fbca48214c7/edit?viewport_loc=-4060%2C-24562%2C22625%2C10832%2C0_0&invitationId=inv_82370c05-f5cd-46a1-8d7f-a6e80739d4c4